

Sensor-Only Wear Prediction on MATWI

Engineered features, per-set deltas, and gradient boosting

Project PINN — Stage 2 sub-track

Abstract

This report documents the sensor-only sub-track of the MATWI wear-prediction project. The starting point was a Ridge regression on 40 hand-engineered sensor features (`sensor_check.py`), which produced a test MAE of $56.0\mu\text{m}$ — *worse* than predicting the training mean ($40.9\mu\text{m}$). A second-generation pipeline (`sensor_only_v2.py`) was built that combines (i) a richer dimensionless feature set, (ii) per-set delta features that subtract a learned “unworn baseline,” (iii) LightGBM in addition to Ridge, and (iv) leave-one-set-out cross-validation. The best resulting method, `lgbm_v2cut_absolute`, reaches **$29.9\mu\text{m}$ test MAE** ($-11m$ vs. predict-mean, $-26m$ vs. the original baseline) with **$54.8\mu\text{m}$ LOSO-pooled MAE**. All LightGBM variants cluster in the $29\text{--}33m$ test band, indicating a genuine ceiling on the engineered-feature sensor-only setup. The dominant remaining error source is two anomalous training sets (5 and 11) whose LOSO folds blow up to $93\text{--}169\mu\text{m}$ regardless of model or features, and which are *not* explained by cutting-parameter differences. The result is a defensible sensor-only baseline; further sensor work shows diminishing returns relative to the project’s main contribution, the Stage 3 physics-informed loss.

1 Motivation and scope

The MATWI dataset (De Pauw et al., KU Leuven, 2023) provides paired images and multi-channel sensor recordings for 17 milling tools run to failure. The project’s main pipeline performs *vision-only* and *vision+sensor fusion* wear prediction. Before invoking fusion, a question worth asking is: **can the sensor channels alone predict wear at all?**

A first attempt (`sensor_check.py`, the predecessor of this sub-track) answered: not really. A Ridge regression on 40 hand-engineered sensor features achieved $56.0\mu\text{m}$ test MAE — a model that performs *worse* than the trivial baseline of always predicting the training mean ($40.9\mu\text{m}$). The training MAE of $21.7\mu\text{m}$ confirmed massive overfitting; the model was memorising set identity, not learning wear.

This sub-track set out to build a meaningful sensor-only baseline. The goal was not to match vision-only performance ($19\mu\text{m}$ test MAE), which is unrealistic on this dataset, but to (a) beat predict-mean by a credible margin, (b) understand *why* the v1 attempt failed, and (c) establish a clean reference for future physics-informed and fusion experiments.

2 Folder contents

All files described below live in this folder (`sensor-only/`). The shared multimodal dataset class is in the sibling `vision-sensor/` folder and is imported via a path-search helper.

`sensor_check.py`

The first-generation sensor-only baseline. Reuses the `MATWIMultimodalDataset` class from `vision-sensor/` to extract its 40 hand-engineered features per sample (5 channels $\times$ 8 features: mean, std, RMS, peak-to-peak, kurtosis, dominant frequency, low-band energy, mid-band energy). Fits a Ridge regression with an L2 standardiser; reports MAE on train, val, test broken down by wear type, and dumps per-feature [coefficient] rankings. Used to diagnose the original failure mode (it overfits to absolute, gain-dependent spectral energies).

`sensor_check.sh`

SLURM submission script for `sensor_check.py`, configured for Snellius (gpu_a100, 30 minutes, 4 CPUs, 16 GB). The CPU budget is the bottleneck because feature extraction is dominated by CSV I/O, not GPU compute.

`sensor_only_v2.py`

The second-generation pipeline and the main contribution of this sub-track. Re-implements the feature extractor with ~ 85 dimensionless features (relative spectral bands, robust spread statistics, within-pass windowed RMS trend, cross-channel force resultant); adds an optional per-set delta transform; supports both Ridge and LightGBM models; loads cutting parameters (`Vc`, `fz`, `Vf`, `Ae`, `Ap`, `z`, `material`) from `sets.csv` as additional features for LightGBM; and runs all comparisons under both the official train/val/test split and leave-one-set-out cross-validation on training sets. Outputs are cached so re-runs take ~ 30 seconds instead of 5 minutes.

`sensor_only_report.tex`

This document.

Results from running `sensor_only_v2.py` are written by default to `runs/sensor_only_v2/` at the project root:

```
runs/sensor_only_v2/
features_cache_1_13.npz extracted v1+v2 features (reused on reruns)
results.json full per-method, per-split metrics
results.csv flat table (sorted by test MAE)
loso_results.csv per-fold leave-one-set-out breakdown
feature_importance.csv top 20 features of the best method
```

3 Methodology

3.1 Diagnosis of the v1 failure

The v1 extractor’s 40 features included *absolute* spectral band energies (`Fx_low_energy` = `np.sum(fft_power[ma`). These quantities scale linearly with sensor gain, signal amplitude, and recording length — all of which differ per set in MATWI. The v1 Ridge model’s top-5 features by `|coef|` were exactly these band energies, meaning the model was using them as set-identity proxies. When the train→test distribution shift kicked in (train is CK45 only; val/test include RVS304), those proxies became anti-correlated with wear and the model collapsed.

Three concrete problems were identified in the v1 extractor:

1. **Absolute spectral energies** (gain- and length-dependent).
2. **Whole-pass global statistics** (`p2p`, `kurtosis`) over $\sim 99,000$ samples, dominated by single outliers; whole-pass dominant frequency, which is essentially meaningless on a broad spectrum.
3. **No cross-channel physical features**, although tool wear physically raises the cutting force resultant $F_{\text{res}} = \sqrt{F_x^2 + F_y^2 + F_z^2}$ in a coordinate-frame-independent way.

3.2 Tier 1 — v2 feature engineering

The new feature extractor produces 85 features per sample:

Per channel (15 features $\times$ 5 channels = 75). For each of `acc`, `acoustic`, `Fx`, `Fy`, `Fz`:

Basic statistics (3)

mean, std, RMS.

Robust spread (2)

interquartile range $p_{75} - p_{25}$ and $p_{95} - p_5$. Replaces the v1 `p2p`, which over 99k samples reduces to a single outlier.

Robust shape (1)

fraction of samples beyond 3σ from the mean. Replaces the v1 `kurtosis`, which is similarly outlier-dominated.

Windowed RMS trend (4)

split the pass into $N=10$ contiguous windows; compute RMS per window; fit a line to RMS vs. normalised window index. Report (mean, std, slope, intercept). The *slope* is the within-pass temporal evolution that whole-pass globals destroy.

Spectral (5)

three *relative* band energies (each band’s power divided by total power), spectral entropy (Shannon, on the normalised power spectrum), and spectral centroid in Hz. All five are dimensionless or in physical units that do not depend on sensor gain or recording length.

Cross-channel (10 features).

- Force resultant F_{res} : mean, std, windowed (mean_rms, std_rms, slope_rms, intercept_rms). Six features.
- Mean of $|F_x|/|F_y|$ ratio.
- Per-axis coefficient of variation $\sigma/|\mu|$ for F_x, F_y, F_z .

The hypothesis was that these features would generalise across setups where the v1 features could not. Empirically, this turned out to be true only for tree-based models (see Results).

3.3 Tier 2 — per-set delta features

The MATWI dataset suffers from setup-to-setup signal-amplitude drift that survives even careful feature engineering. To remove this for linear models, the v2 pipeline computes a *per-set baseline feature vector* as the mean over the $K=3$ lowest-`SensorID` passes for each set, then subtracts that baseline from every sample in the set:

$$\tilde{x}_i = x_i - \frac{1}{K} \sum_{j \in \text{first-}K(s_i)} x_j.$$

The key observation: `SensorID` is the recording-order index *not* the wear label, so using the early-passes-as-reference is legitimate even on val/test sets — there is no label leakage. Each set is also entirely contained in one official split (e.g., all of set 3 is in val), so the baseline computation does not cross the split boundary.

Per-set deltas dramatically improved Ridge (test MAE 56.0→34.8 for the v1 features) but slightly hurt LightGBM, because tree splits do not benefit from mean-centering — subtracting a baseline destroys absolute-scale information that splits could otherwise use.

3.4 Tier 3 — LightGBM

LightGBM (Microsoft’s gradient-boosted decision-tree implementation) is the natural model class for tabular problems of this size. Compared to Ridge, it captures nonlinear feature interactions and is invariant to feature scaling (tree splits are threshold comparisons). Compared to neural networks, it is far less prone to overfitting on ~ 647 training samples and provides interpretable per-feature gain importances.

Hyperparameters (small-data conservative):

```
objective = "regression_l1" # MAE objective
learning_rate = 0.03
num_leaves = 31
max_depth = 5
min_data_in_leaf = 10
feature_fraction = 0.8 # per-tree feature subsample
bagging_fraction = 0.8 # per-tree row subsample
lambda_l2 = 1.0 # L2 on leaf values
num_boost_round = 3000
early_stopping = 150 # patience on val MAE
```

Missing values (e.g., Set 1’s unknown cutting parameters) are passed through natively — LightGBM learns at each split whether NaN values should be routed left or right.

The original v1 `sensor_check.py` additionally provided a **Ridge sanity check** that this sub-track did not replace. Treating that script as the v1 baseline keeps the comparison honest: both v1 Ridge and v2 Ridge are evaluated in `sensor_only_v2.py` under identical splits and evaluation harnesses.

3.5 Cutting-parameter features

The eight cutting parameters from `sets.csv` (V_c , n , f_z , V_f , A_e , A_p , z , material as a binary RVS flag) are appended to the v2 features for the LightGBM `*cut*` methods. The csv has a few messy cells: Set 1 has all-unknown parameters (encoded as “?”) and Set 6 has variable feed rate “185/148” — both handled by the parser ($? \rightarrow \text{NaN}$, “a/b” $\rightarrow$ mean of parts).

The hypothesis was that adding cutting parameters would let LightGBM *route* predictions by cutting regime, especially for the LOSO-outlier sets 5 and 11. As reported below, this hypothesis failed empirically.

3.6 Evaluation: official split + leave-one-set-out CV

The standard evaluation is on the official train (sets {1,2,5,7,8,10,11}) / val ({3,6,12}) / test ({4,9,13}) split, where wear MAE is also broken down by wear type (flank, adhesion, flank+adhesion). However, the official val set contains set 12 (RVS 304, never seen in training) and is therefore an unreliable guide to in-distribution generalisation.

Leave-one-set-out (LOSO) CV on the seven training sets provides a fairer signal. For each training set s , train on the other six sets and evaluate on s . Predictions are pooled across folds and the pooled MAE is reported. For delta-based methods, the per-set baseline is recomputed inside each fold (using the held-out set’s own `SensorID` order, so there is no label leakage).

The discrepancy between LOSO MAE and test MAE is itself diagnostic: similar values mean the model generalises across setups; a much worse LOSO indicates dependence on specific training sets.

4 Results

4.1 Comparison matrix

Table ?? summarises all methods on the official split plus LOSO-pooled CV on training sets.

Table 1: Test, val and LOSO-pooled MAE in μm . The full method matrix from `results.csv`.

Method	val	test	LOSO	train
predict_mean_train	57.7	40.9	—	—
ridge_v1_absolute (baseline)	85.8	56.0	78.7	21.7
ridge_v1_delta	78.4	34.8	112.7	23.0
ridge_v2_absolute	84.2	59.2	106.1	19.9
ridge_v2_delta	88.6	50.1	69.0	18.7
lgbm_v2_absolute	49.9	31.9	55.0	49.3
lgbm_v2_delta	49.7	35.0	62.7	54.5
lgbm_v2cut_absolute	50.2	29.9	54.8	44.8
lgbm_v1v2_absolute	49.8	31.2	58.3	49.1
lgbm_v1v2cut_absolute	49.9	32.6	60.9	51.3

Five observations:

1. **Best method: lgbm_v2cut_absolute at 29.9 μm test, 54.8 μm LOSO.** A clean -11 m vs. predict-mean and -26 m vs. the original Ridge baseline.
2. **LightGBM variants cluster tightly (29.9–32.6 μm test).** Cutting parameters, kitchen-sink feature unions, and patience tweaks all move the needle by less than 3 m . The model is at its ceiling.
3. **Per-set delta helps Ridge but hurts LightGBM.** Ridge needs the linear-model fix; tree splits handle scale natively and lose information when it is removed.
4. **The v2 features helped only when combined with LightGBM.** Under Ridge, v2 was no better than v1 (in fact slightly worse) — 85 features on 647 samples overfits Ridge. Under LightGBM, the new robust-spread, spectral, and cross-channel features are heavily used (see §??).
5. **Val is harder than test.** Set 12 (RVS304) makes val a true out-of-distribution split. Best-LightGBM val MAE (49.9 μm) is barely below predict-mean on val (57.7 μm), while test MAE drops by $\sim 11\text{ }\mu\text{m}$. The official val set is therefore unreliable for selecting between models; LOSO is the better signal.

4.2 Feature importance (best method)

Table ?? lists the top 20 features of `lgbm_v2cut_absolute` by LightGBM “gain” importance. The picture is consistent and physical: force channels dominate (17 of the top 20), the force resultant cross-channel feature is used, and the v2-specific robust spread features ($p_{95}-p_5$) hold the top positions. Cutting parameters were available but not heavily used, explaining why they added almost no test-MAE improvement.

4.3 Per-fold LOSO breakdown

Pooled LOSO MAE hides important structure. Table ?? shows per-fold MAE for the two best methods.

Five of seven folds give 25–57 μm — a perfectly respectable sensor-only result. The remaining two (5 and 11) blow up by a factor of 3–6 $\times$, dragging the pooled LOSO from $\sim 28\text{ }\mu\text{m}$ (if those two sets were excluded) up to 55 μm . Crucially, adding cutting parameters does *not* fix sets 5 and 11; the largest improvement on either is -1.7 m . Sets 5 ($V_c=174, f_z=0.04, V_f=148$) and 11 ($V_c=174, f_z=0.043, V_f=159$) sit *inside* the parameter envelope of the other training sets, so they are not extrapolating in regime space. The anomaly must therefore be in the sensor signatures

Table 2: Top 20 features by LightGBM gain importance. Eight of the top 20 are v2-specific (marked *); all ten cutting parameters were available but only one (**material_RVS**, not shown) appeared in the top 30.

Rank	Feature	gain
1	Fx_p95mp05*	1047
2	Fy_std	863
3	Fx_std	725
4	Fy_p95mp05*	242
5	Fy_spec_entropy*	160
6	Fy_mean	155
7	Fx_mean	121
8	Fx_cv*	106
9	Fx_spec_entropy*	86
10	Fy_rel_e_b2*	70
11	Fy_cv*	66
12	Fx_win_mean_rms*	57
13	acc_frac3sig*	57
14	Fres_win_std_rms*	56
15	Fy_win_mean_rms*	49
16	Fy_win_intercept_rms*	46
17	Fz_mean	36
18	acoustic_frac3sig*	34
19	acc_win_std_rms*	32
20	FxFy_ratio_mean*	30

Table 3: Leave-one-set-out per-fold test MAE in μm . Sets 5 and 11 are systematic outliers — adding cutting parameters barely changes them.

Heldout set	lgbm_v2_absolute	lgbm_v2cut_absolute	Δ
1	27.9	26.2	−1.7
2	56.0	57.6	+1.6
5	168.9	168.5	−0.4
7	25.3	27.1	+1.8
8	25.4	26.7	+1.3
10	28.9	25.9	−3.0
11	94.4	93.4	−1.0

or wear-progression curves themselves — likely tool variation, lubrication, or sensor calibration drift on those recordings — and is not recoverable through feature engineering on the existing channels.

5 Findings

1. The original 56 μm Ridge failure was caused by the extractor relying on *absolute* spectral band energies that doubled as set-identity proxies. Replacing them with dimensionless spectral shape features (relative band energy, entropy, centroid) fixed this in principle, but the gain only materialised once the model class was changed too.
2. **The single biggest intervention for sensor-only is switching to LightGBM.** Ridge with v2 features peaks at 50 μm test; LightGBM with the same features peaks at 32 μm . Tree splits both (a) capture nonlinear interactions between features that Ridge cannot, and (b) handle the residual setup-to-setup scale drift that Ridge stumbles on.
3. **Per-set delta features are a Ridge-specific patch, not a universal one.** They cut

Ridge test MAE on v1 features by $\sim 21\mu\text{m}$ but cost LightGBM $\sim 5\mu\text{m}$. This is a useful lesson for any future modelling: linear-model fixes do not always transfer to tree models.

4. **Cutting parameters helped slightly but did not explain the LOSO outliers.** The hypothesis that sets 5 and 11 were anomalous because of their cutting regime was *falsified*: the model can see the parameters and still fails on those folds. The actual cause is at the sensor or wear-progression level, not the cutting physics.
5. **There is a real ceiling.** All LightGBM variants are within $3\mu\text{m}$ of each other on test and within $6\mu\text{m}$ on LOSO. The kitchen-sink combination (v1Uv2Ucutting) is in fact the worst LightGBM variant, indicating diminishing returns from feature accumulation. Further engineering on the same sensor channels is unlikely to move the needle.

6 Limitations and the ceiling

The remaining error budget breaks down approximately as follows (qualitative, not from a formal decomposition):

- **Two anomalous training sets (5, 11)** account for the gap between pooled LOSO ($\sim 55\mu\text{m}$) and the typical-fold LOSO ($\sim 28\mu\text{m}$). These would require either dropping (yielding a more optimistic but smaller-data estimate) or a fundamentally different model class — per-sample rather than per-pass features, for instance.
- **CK45 $\rightarrow$ RVS 304 distribution shift** caps val and part of test performance. Set 12 (val) and set 13 (test) are RVS304 but no RVS304 appears in train under the official `set_range="1-13"` setting. Running with `-set-range 1-17` (adding sets 14–17 to training) would address this but breaks comparability with the published baselines.
- **Sample size (647 training samples)** caps model complexity. Approaches like 1D-CNNs over raw downsampled sequences, or transformer encoders over windowed segments, would add learned representations on top of the engineered features, but each additional parameter risks overfitting on a corpus this small.
- **Vision-only is structurally better-suited to this task.** The vision-only baseline reaches $19\mu\text{m}$ test MAE on the same split because tool wear is fundamentally a visual phenomenon (a worn flank looks different from a fresh one), while the sensor signals reflect wear only indirectly through forces and vibrations that depend on many other factors. **Sensor-only is not expected to match vision-only on this dataset**, and reaching $29.9\mu\text{m}$ ($1.6\times$ vision-only) on a per-feature-engineering pipeline is a respectable result.

7 How to reproduce

```
# From the project root, after the dataset is downloaded into
# dataset/matwi/ via dataset/download_data.py.

# Optional but recommended for the full method matrix:
pip install lightgbm

# Run all methods, save outputs under runs/sensor_only_v2/
python sensor-only/sensor_only_v2.py \
  --data-dir dataset/matwi \
  --labels-csv dataset/matwi/labels.csv \
  --output-dir runs/sensor_only_v2
```

```
# First run extracts features (~5 min, dominated by CSV I/O).  
# Subsequent runs reuse the cache and finish in ~30 s.  
  
# To exclude cutting-parameter methods:  
# --no-cutting-params  
  
# To use all sensor samples (drops the require-image filter that  
# matches the existing 647-sample baseline):  
# --all-sensor-samples  
  
# To include RVS 304 (sets 14-17) in training:  
# --set-range 1-17
```

8 Conclusion

The sensor-only sub-track produced a credible baseline (**29.9 μm test, 54.8 μm LOSO**) and a clear diagnosis of why it cannot be pushed materially further on the existing sensor channels: two anomalous training sets account for most of the LOSO error, and the distribution shift to RVS304 caps val performance. This is enough to support the project’s main argument — that physics-informed loss (Stage 3) is a better lever than additional sensor work — and to provide a clean reference point for the sensor-fusion comparisons in the multimodal sub-track.